

RMIT University
Cloud Computing — Assessment 3

SOLUTION ARCHITECTURE
**OpenPortfo: A Cloud-Based Crypto and Vietnam
Stock Portfolio Tracking System**

Student ID: s3927049
Student name: Mai Gia Phu
September 2026

Table of Contents

1. Links.....	3
2. Summary.....	3
3. Introduction.....	3
3.1 Motivation.....	3
3.2 High-level view.....	3
3.3 Beneficiaries.....	4
4. Related work.....	4
5. System Architecture.....	5
5.1 Architectural diagrams.....	5
5.2 System description.....	8
Compute.....	8
Storage.....	9
Networking and Content Delivery.....	9
Database.....	9
Analytics.....	9
Application Integration.....	10
Security, Identity, and Compliance.....	10
Client interface and third-party APIs.....	10
Why this shape.....	11
5.3 Datasets, data structures and APIs.....	11
Datasets.....	11
Data structures.....	12
Application APIs.....	12
6. References.....	13

1. Links

The application is deployed on an AWS Academy Learner Lab account in us-east-1.

Live application URL: <https://openportfo-api-env.eba-yrwmpggu.us-east-1.elasticbeanstalk.com> (required https for Cognito to work, may require to approve)

REST API Gateway: <https://7duvngr98b.execute-api.us-east-1.amazonaws.com> (route ANY /api/{proxy+} → Elastic Beanstalk FastAPI)

Source code repository: <https://github.com/maiphh/openportfo>

Cognito Hosted UI: <https://openportfo-11f1.auth.us-east-1.amazoncognito.com>

Data: Market prices come from CoinGecko and vnstock at run time. News items come from public RSS feeds configured on the Admin page. Daily portfolio snapshots are generated by the application and stored in S3 for Athena.

2. Summary

OpenPortfo is a web application that lets one person track cryptocurrency and Vietnam-listed stock holdings in a single place. The user signs in, searches assets, saves a watchlist and holdings with quantity and cost, and sees market value, profit and loss, allocation charts, and a history chart. Prices are loaded on demand and cached; they are not streamed. A daily Lambda job stores a portfolio snapshot, pulls RSS news that matches the user's symbols, and can send a summary email. An admin page controls RSS sources, job flags, cache time, and USD/VND rates.

The live cloud split is simple. The browser loads the Next.js static UI from Elastic Beanstalk. REST calls under /api/* go to Amazon API Gateway, which forwards them to FastAPI on the same Beanstalk environment. Chat streaming stays on Beanstalk because the Gateway 30-second timeout is too short. Scheduled work is EventBridge → Lambda only. DynamoDB holds live records. S3 holds history and snapshot files. Athena runs SQL on those snapshot files.

3. Introduction

3.1 Motivation

Crypto wallets and Vietnam broker apps do not share a portfolio view. People end up with two screens plus a spreadsheet, so total value, allocation, and profit and loss are hard to see together. Paid multi-asset products are usually built for US or global tickers, not HOSE names next to Bitcoin. A student cloud project also has a hard spend cap, so a real-time exchange or a large always-on cluster is the wrong shape.

OpenPortfo is built to close that gap with a small AWS stack: one login, one dashboard, on-demand prices, keyword-matched news, and an optional daily email, without executing trades.

3.2 High-level view

At a high level the product is a personal portfolio tracker, not a broker. After login the user can search Bitcoin or a Vietnam stock such as VNM, add it to a watchlist or as a holding, and open a dashboard that shows value, cost, PnL, a pie chart, and a table. Asset pages show a history chart. A news list is filled by a nightly job. A chat bubble can answer

questions about the user's own holdings. An admin user can change RSS feeds, turn jobs on or off, and refresh FX rates.

Two traffic paths stay separate on purpose. User clicks go through the browser, API Gateway (for REST), and FastAPI. Nightly work is EventBridge calling Lambda. The browser never talks to CoinGecko, vnstock, or DynamoDB itself.

3.3 Beneficiaries

Individual investors. People who hold both crypto and Vietnam stocks and want one dashboard instead of two independent apps and a sheet.

4. Related work

Several products already track portfolios, but they split along asset-class lines. Table 1 places OpenPortfo next to tools that students and retail investors use.

Product	Crypto	VN stocks	Unified view	News	Email / chat	Deploy model
CoinMarketCap Portfolio	Yes	No	Crypto only	Site news	Limited	SaaS web
Delta / Blockfolio-style apps	Yes	No	Crypto only	Alerts	Push	Mobile SaaS
Yahoo Finance	Limited	Limited VN	Global multi-asset	Yes	Alerts	SaaS
CafeF / TCBS tools	No	Yes	VN equities	Market news	Varies	Local portals
Google Sheets trackers	Manual	Manual	If the user builds it	No	No	DIY
OpenPortfo (this work)	Yes	Yes	Crypto + VN	RSS + keywords	Daily email + chat	AWS self-deploy

Table 1. Related products compared with OpenPortfo.

CoinMarketCap and Delta-class apps are strong for crypto and weak for Vietnam broker holdings. Yahoo Finance covers many global markets, but Vietnam symbols and a free automated digest are not the point of that product. CafeF and TCBS tools are the usual place for HOSE/HNX prices and local news; they do not value a crypto wallet next to those stocks. Spreadsheet trackers can do anything once, then rot.

OpenPortfo is not a full brokerage and not a production fintech product. It is a complete student AWS system: dual market APIs, Beanstalk user APIs versus Lambda schedules, admin-managed RSS, Athena-ready snapshots, and a Cognito login, mainly focus on tracking asset values.

5. System Architecture

5.1 Architectural diagrams

Figure 1 is the whole system. Table 2 and Figure 2 show how a click in the UI reaches other components. Figure 3 is the nightly path, which the browser never starts. Table 4 is the data layout.

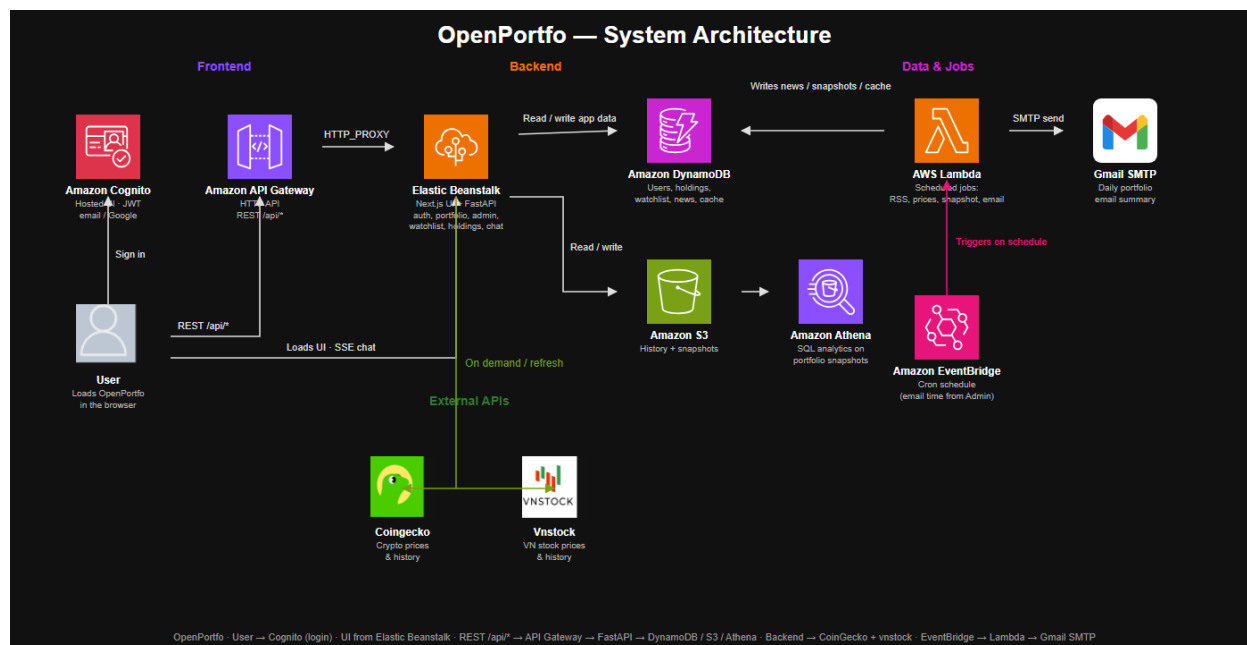


Figure 1. OpenPortfo high-level architecture (AWS service icons). Each icon states the component function.

The intended production UI path is CloudFront in front of an S3 website bucket. AWS Academy blocks CloudFront in this lab, so the live demo serves the same Next.js static export from Elastic Beanstalk. S3 is still used for history and snapshot files. REST still uses API Gateway. That lab constraint does not change the two-path design: user requests versus EventBridge jobs.

UI action	Entry	Then	Stores / providers
Open site / click pages	EB static files	Browser renders Next.js CSR	No AWS write
Sign in / Sign up / Google	Cognito Hosted UI	Callback → ID token on later calls	Cognito; DynamoDB profile
Search assets	Gateway → FastAPI	GET /api/assets/search	CoinGecko or vnstock
Add / edit holding or watchlist	Gateway → FastAPI	POST/PUT/DELETE holdings or watchlist	DynamoDB
Open portfolio dashboard	Gateway → FastAPI	GET /api/portfolio (cache first)	DynamoDB + PriceCache
Click Refresh prices	Gateway → FastAPI	POST /api/portfolio/refresh	CoinGecko + vnstock → cache

Open history chart	Gateway → FastAPI	GET /api/assets/.../history	S3 cache or live fetch
Read news	Gateway → FastAPI	GET /api/news (read only)	DynamoDB News (filled by Lambda)
Ask chatbot	EB same-origin SSE	POST /api/chat/stream (not Gateway)	OpenRouter; DynamoDB idempotency
Admin: RSS, jobs, FX refresh	Gateway → FastAPI	Admin APIs; FX calls ExchangeRate-API	DynamoDB settings / FX
Export CSV	Gateway → FastAPI	GET /api/portfolio/export	Same valuation as dashboard

Table 2. How each client-interface operation invokes the rest of the system.

Table 2 is the map that show how client interact with the system: each UI action, the first AWS entry, the FastAPI call, and what is read or written. Three rules stay fixed. The browser does not call market APIs. Lambda is not a public HTTP API.

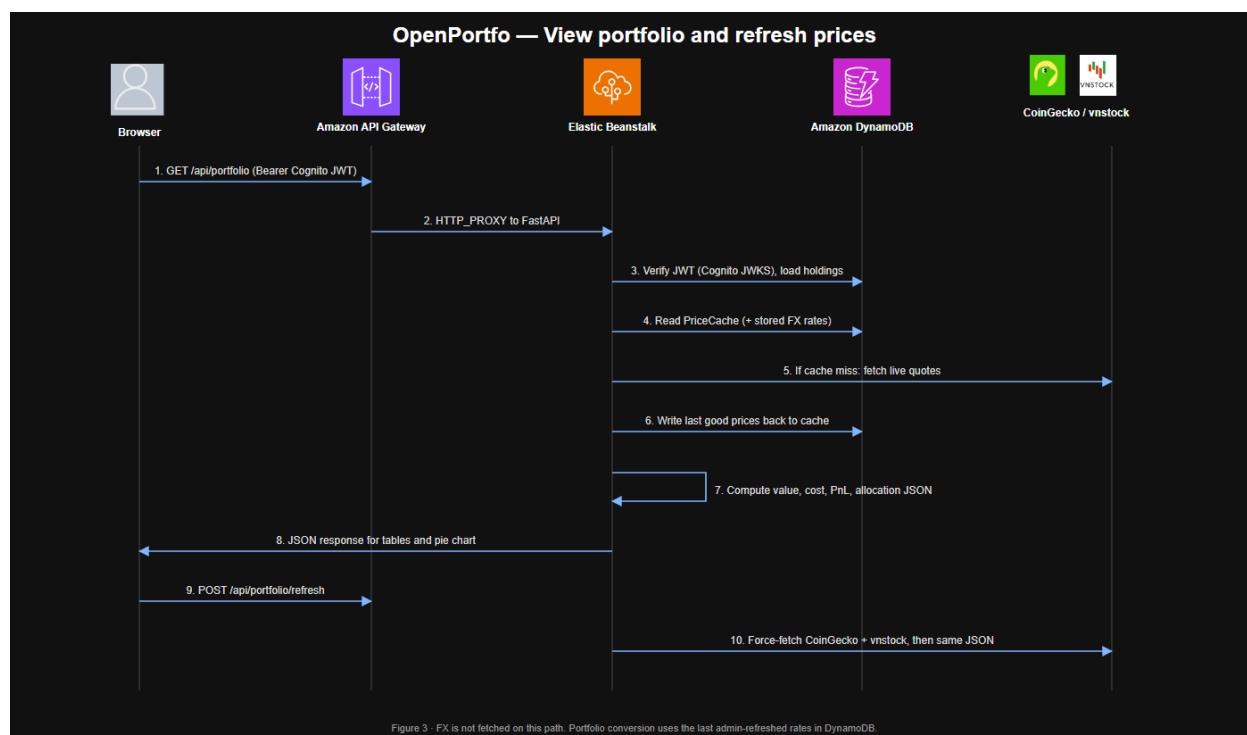


Figure 2. Sequence for view-portfolio and Refresh. This is the main request-path interaction.

On a warm cache, GET /api/portfolio is FastAPI plus DynamoDB only. Refresh, or a cache miss, adds CoinGecko and vnstock, then writes PriceCache so the next view is cheap. FX is not fetched here. Conversion uses the last rates an admin stored.

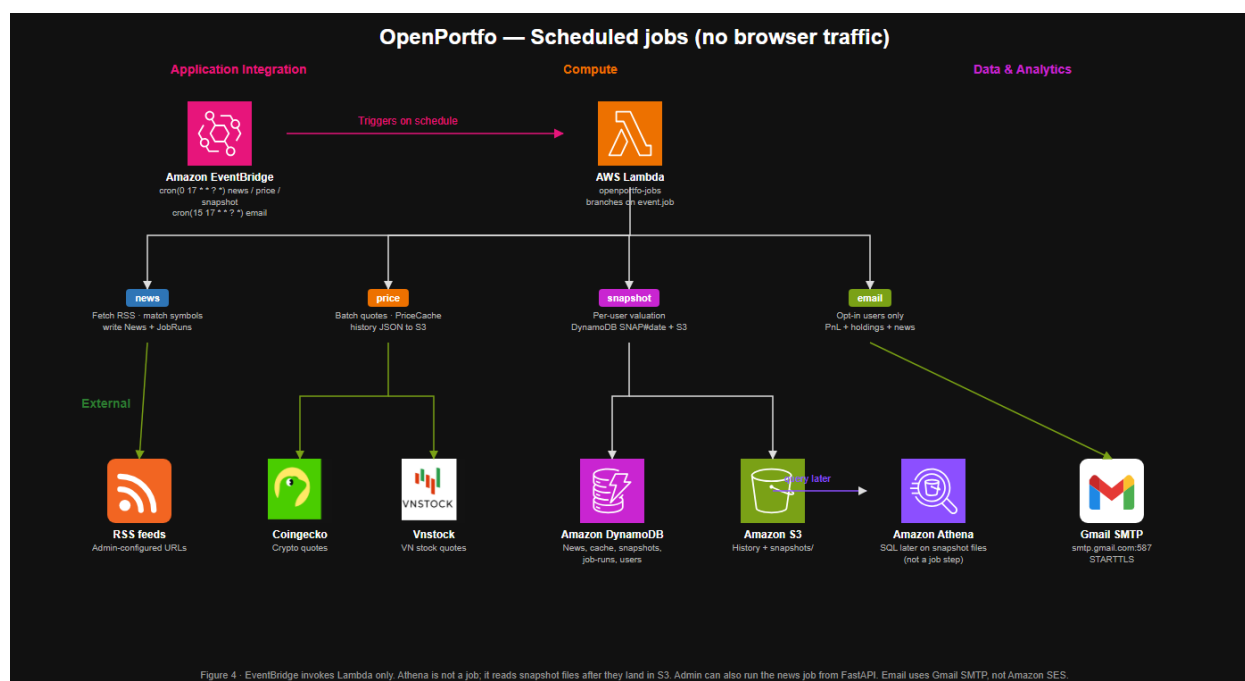


Figure 3. Schedule path: EventBridge invokes Lambda. Four jobs share one function and branch on event.job. Email leaves AWS through Gmail SMTP.

EventBridge rules in us-east-1 run news, price, and snapshot at 17:00 UTC (00:00 UTC+7) and email at 17:15 UTC (00:15 UTC+7). The Lambda name is openportfo-jobs. Admin can also run the news job from the UI; that is still FastAPI invoking Lambda, not the browser calling Lambda. Athena is not a fifth job: it reads snapshot files after they already sit in S3.

Job	Trigger	Time	Action
news	EventBridge cron (admin can also run it from the UI)	Daily 17:00 UTC (00:00 ICT)	Fetch admin RSS URLs, keyword / symbol match, write News + JobRuns
price	EventBridge cron	Daily 17:00 UTC (00:00 ICT)	Batch quotes for known symbols → PriceCache (+ history to S3)
snapshot	EventBridge cron	Daily 17:00 UTC (00:00 ICT)	Per-user valuation → DynamoDB SNAP#date + S3 snapshots/userId=dt=
email	EventBridge cron	Daily 17:15 UTC (00:15 ICT)	PnL + holdings + matched news to opt-in users via Gmail SMTP

Table 3. Scheduled jobs: trigger, time, and action.

Table 4 is the data design: every DynamoDB table, its keys, and the S3 prefixes Athena reads. userId is always the Cognito sub. The table prefix in the lab is openportfo-.

Store	Name	Keys	Main attributes	Written by
DynamoDB	users	PK userId	email, name, role, keywords, emailOptIn, currency	FastAPI /auth/me
DynamoDB	holdings	PK userId, SK HOLD#type#symbol	qty, avgCost, currency, note	Holdings API
DynamoDB	watchlist	PK userId, SK WATCH#type#symbol	addedAt	Watchlist API

DynamoDB	price-cache	PK type#symbol	price, currency, asOf, ttl	FastAPI + price job
DynamoDB	news	PK date, SK source#hash	title, url, symbols, keywords	News Lambda
DynamoDB	snapshots	PK userId, SK SNAP#date	lines, totals, fx	Snapshot Lambda
DynamoDB	settings	PK SETTINGS, SK GLOBAL	jobs, cache TTL, chat model	Admin API
DynamoDB	fx	PK FX, SK LATEST	rates, asOf, lastRefreshStatus	Admin FX refresh
DynamoDB	rss	PK RSS, SK sourceId	name, url, enabled	Admin RSS API
DynamoDB	job-runs	PK JOB#type, SK runAt	status, message, counts	Lambda / admin run
DynamoDB	chat-idempotency	PK userId, SK requestId	expiresAt (TTL)	Chat API
S3	snapshots/	userId={id}/dt={YYYY-MM-DD}/part.json	lines, totalsByCurrency, fx	Snapshot Lambda
S3	history/	{crypto stock}/{id}/{range}.json	OHLCV / market_chart cache	FastAPI + price job
S3	athena-results/	query output prefix	Athena result files	Athena

Table 4. DynamoDB tables and S3 object layout used by Athena.

5.2 System description

OpenPortfo uses AWS services under Compute, Containers, Storage, Networking and Content Delivery, Database, and Analytics. Tables 5–9 use those console categories. Containers are unused: Elastic Beanstalk is the Compute host. Tables 10–12 cover extra AWS services and the third-party APIs the live system calls. Each row states the component function and who invokes it.

Compute

Component	Function	Invoked by	Talks to
Elastic Beanstalk [2]	Hosts FastAPI (JWT check, CRUD, portfolio maths, chat SSE) and, in this lab, the Next.js static UI plus /health	Browser (UI pages and POST /api/chat/stream); API Gateway HTTP_PROXY for REST /api/*	DynamoDB, S3, Cognito JWKS, CoinGecko, vnstock, OpenRouter; Lambda only for admin “run news”
AWS Lambda openportfo-jobs [9]	One function, four jobs: news (RSS match), price (warm cache), snapshot	EventBridge cron; FastAPI admin POST to run the news job	RSS URLs, CoinGecko, vnstock, DynamoDB, S3, Gmail SMTP. Not a

	(value every user), email (PnL digest)		public HTTP API
--	---	--	-----------------

Table 5. Compute services.

Storage

Component	Function	Invoked by	Talks to
Amazon S3 [6]	Data bucket for price-history JSON, daily snapshot files, and Athena query output. Not a public website bucket	FastAPI (history cache) and the price / snapshot Lambda jobs	Athena reads the snapshots/ prefix. Intended UI hosting on S3+CloudFront is blocked in Academy, so the UI is on Beanstalk

Table 6. Storage services.

Networking and Content Delivery

Component	Function	Invoked by	Talks to
Amazon API Gateway HTTP API [3]	REST front door. Proxies ANY /api/{proxy+} with HTTP_PROXY. CORS allows the Beanstalk UI origin. Not an authorizer	Browser REST clients (execute-api hostname)	Elastic Beanstalk FastAPI. Does not front /, /_next/*, /health, chat SSE, or Lambda
Amazon CloudFront	Designed as the UI CDN in front of an S3 website bucket	Not live in this lab — AWS Academy blocks CloudFront	Would fetch static Next.js files from S3. Lab substitute: Beanstalk serves the same export

Table 7. Networking and Content Delivery services.

Database

Component	Function	Invoked by	Talks to
Amazon DynamoDB [5]	Operational store (on-demand): users, holdings, watchlist, price-cache (TTL), news, snapshots, settings, fx, rss, job-runs, chat-idempotency. Keys in Section 5.3. Passwords are never stored	FastAPI adapters on every authenticated request; Lambda jobs for news, cache, snapshots, job-runs	Domain code does not import boto3. userId is the Cognito sub

Table 8. Database services.

Analytics

Component	Function	Invoked by	Talks to
Amazon Athena [7]	SQL on snapshot files (database openportfo, table portfolio_snapshots_raw, JSON SerDe). Not on the dashboard hot path	Saved query openportfo-latest-snapshots after the snapshot job has written S3. Not EventBridge and not the browser	S3 snapshots/ prefix; results land under athena-results/ so scans stay small

Table 9. Analytics services.

Application Integration

Component	Function	Invoked by	Talks to
Amazon EventBridge [8]	Clock for unattended jobs. Four ENABLED rules: news, price, snapshot at cron(0 17 * * ? *); email at cron(15 17 * * ? *)	Schedule only (00:00 / 00:15 ICT). The browser never calls EventBridge	Lambda openportfo-jobs with {"job": "news" "price" "snapshot" "email"}

Table 10. Application Integration services.

Security, Identity, and Compliance

Component	Function	Invoked by	Talks to
Amazon Cognito [4]	User Pool us-east-1_YRFHh9x1r. Hosted UI for email/password and optional Google. Issues ID tokens. SPA is a public app client	User Sign in / Sign up / Google from the browser	Browser keeps the ID token and sends Authorization: Bearer. FastAPI verifies JWKS (signature, issuer, audience, expiry)

Table 11. Identity services.

Client interface and third-party APIs

Component	Function	Invoked by	Talks to
Next.js static CSR [12]	Markets, watchlist, portfolio, asset detail, settings, admin, Cognito callback. Charts and tables render from FastAPI JSON. No Next.js server on AWS	User in the browser	REST via API Gateway; UI files and chat SSE from Elastic Beanstalk. Never CoinGecko or DynamoDB directly
Gmail SMTP	Daily portfolio email (PnL, holdings, matched news) to opt-in users. Academy denies SES, so the live	Lambda email job at 00:15 ICT when admin jobs.email and user emailOptIn are on	Gmail only. App Password stays in Lambda env, not in Git

	path is smtp.gmail.com:587 STARTTLS. SES is not used		
CoinGecko [1]	Graded third-party API. Crypto search, USD quotes, market_chart history	FastAPI on search / quote / refresh / history; price Lambda	Results cached in DynamoDB PriceCache and history objects in S3
vnstock [10]	Graded third-party API. HOSE/HNX search, last price, OHLCV	FastAPI on search / quote / refresh / history; price Lambda	Same cache path as CoinGecko. Last good price kept if a fetch fails
Public RSS	News ingest. Not a vendor SDK; Lambda GETs feed XML from admin URLs	News Lambda (cron or admin run)	Items matching symbols or keywords written to the news table
OpenRouter [14]	Chat LLM. FastAPI sends holdings as context and streams the reply	POST /api/chat/stream on Beanstalk (not Gateway)	DynamoDB chat-idempotency TTL only; no transcript store
ExchangeRate-API [11]	USD/VND rates. Admin on-demand only, not the portfolio hot path	POST /api/admin/fx/refresh	Stored in the fx table. A failed refresh keeps the last good rates

Table 12. Client interface and third-party APIs.

Why this shape

User traffic and background work are split so each AWS service has one job. Elastic Beanstalk, API Gateway, and Lambda are invoked by the UI, by FastAPI, or by EventBridge — not only from the console. DynamoDB, S3, and Athena cover records, files, and SQL. Cognito is the login service. EventBridge is the clock. Daily mail uses Gmail SMTP because Academy blocks SES. Cost stays under the USD \$50 cap by using one small Beanstalk instance, on-demand DynamoDB, Lambda free tier, and Athena queries that scan snapshot JSON rather than large scans.

5.3 Datasets, data structures and APIs

Datasets

There is no downloaded CSV that the marker must install. Three data sources feed the system, and one dataset is produced by it.

- **CoinGecko (live).** Crypto instruments, current USD prices, and market_chart history. Free-tier limits are the reason for PriceCache TTL and batch quotes.
- **vnstock / public broker APIs (live).** Vietnam listed symbols, last price, and OHLCV. These endpoints can change; FastAPI keeps the last cached price if a fetch fails.
- **Public RSS (live).** Feed URLs are stored in the rss table and edited on /admin. Items are not a file in the repo. Matching is substring on title against symbols and user keywords.

- **Generated snapshots (S3).** Each night (or on demand) the snapshot job writes part.json per user and date. That object is the Athena dataset. Fields: userId, date, lines[], totalsByCurrency{}, fx{status, base, asOf, rates}.

Data structures

Table 4 (Section 5.1) is the DynamoDB and S3 layout used in the lab. Physical tables are separate on purpose so the demo is easy to walk through. userId is always the Cognito sub. Portfolio maths are done in FastAPI, not in Athena:

marketValue = $\sum (qty \times price)$; costBasis = $\sum (qty \times avgCost)$; pnl = marketValue – costBasis; allocation = lineValue / marketValue. Crypto lines are primarily USD; VN stocks are VND; display conversion uses stored FX rates only.

Application APIs

Table 13 lists the FastAPI routes the UI actually calls. Unless noted, they are reached as <https://{api-id}.execute-api.us-east-1.amazonaws.com/api/...> after Gateway proxy. Chat stream is an exception: same Beanstalk origin.

Group	Method and path	Auth	What it does
Health	GET /health	None	Beanstalk health check
Auth	GET /api/auth/me	JWT	Upsert profile; return role and settings
Search	GET /api/assets/search	JWT	CoinGecko or vnstock search
Quote	GET/POST /api/assets/{type}/{symbol}/quote[/refresh]	JWT	Cached quote; optional live refresh
Watchlist	GET/POST/DELETE /api/watchlist	JWT	User watchlist CRUD
Holdings	GET/POST/PUT/DELETE /api/holdings	JWT	Positions with qty and cost
Portfolio	GET /api/portfolio	JWT	Value, PnL, allocation JSON
Refresh	POST /api/portfolio/refresh	JWT	Force CoinGecko + vnstock
Export	GET /api/portfolio/export	JWT	CSV of the same valuation
History	GET /api/assets/{type}/{symbol}/history	JWT	S3 cache or live OHLCV
News	GET /api/news	JWT	Read news written by Lambda
FX	GET /api/fx/rates	JWT	Read stored rates only
FX admin	POST /api/admin/fx/refresh	Admin	Call ExchangeRate-API
Admin	GET/PUT /api/admin/settings	Admin	Jobs, TTL, chat model
RSS	CRUD /api/admin/rss-sources	Admin	Feeds for the news job

Jobs	GET /api/admin/job-runs; POST .../jobs/news/run	Admin	Status; manual news invoke
Chat	POST /api/chat/stream	JWT	SSE to OpenRouter; not Gateway
Snapshots	GET /api/snapshots[/{date}]	JWT	Read stored daily snapshots

Table 13. FastAPI endpoints used by the client.

Third-party HTTP APIs called from adapters (never from the browser): CoinGecko REST, vntstock/broker HTTP, ExchangeRate-API Standard/Pair, OpenRouter chat completions, Gmail SMTP, and RSS GET on admin URLs. Google OAuth is handled inside Cognito, not in application code.

5.4 Extra features

A chat bubble answers questions about the user's own holdings. The browser calls POST /api/chat/stream with server-sent events on the Beanstalk origin; chat does not go through API Gateway because its 30-second timeout is too short for streaming. FastAPI sends holdings as context to a free OpenRouter model, all server-side. The model can call tools that perform live actions on the user's data — search assets, get quotes, read the portfolio and holdings, add or remove holdings and watchlist items, read matched news, and run asset or portfolio analysis — in a complete → tools → complete loop. A per-session cap and a DynamoDB chat-idempotency record guard cost; no chat transcript is stored.

A daily email sends the portfolio digest — value, PnL, holdings, and matched news — to opt-in users. EventBridge fires the email job at 17:15 UTC (00:15 ICT); it runs only when the admin jobs.email flag and the user's emailOptIn are on. Academy blocks SES, so delivery uses Gmail SMTP with an app password kept in Lambda environment, never in Git.

5.5 Limitations

Market data rides on free tiers. CoinGecko rate-limits quotes and history, so the app batches requests, caches in DynamoDB/S3, and keeps the last good price on failure; a cold Refresh can still load slowly. vntstock wraps unofficial broker endpoints that can change without notice. The chat model is a free OpenRouter model with low usage limits: answers can be slow and are not always smart, and heavy use can hit the quota.

The lab blocks part of the designed stack. CloudFront distributions cannot be created, so the demo serves the Next.js static export from the Beanstalk API process instead of S3 + CloudFront (S3 remains the data bucket for history and snapshots). SES identities cannot be verified, so daily mail goes through Gmail SMTP as a workaround.

6. References

- [1] CoinGecko, "API documentation." [Online]. Available: <https://www.coingecko.com/en/api/documentation>
- [2] Amazon Web Services, "AWS Elastic Beanstalk Developer Guide." [Online]. Available: <https://docs.aws.amazon.com/elasticbeanstalk/latest/dg/Welcome.html>
- [3] Amazon Web Services, "Amazon API Gateway Developer Guide — HTTP APIs." [Online]. Available: <https://docs.aws.amazon.com/apigateway/latest/developerguide/http-api.html>
- [4] Amazon Web Services, "Amazon Cognito Developer Guide." [Online]. Available: <https://docs.aws.amazon.com/cognito/latest/developerguide/what-is-amazon-cognito.html>
- [5] Amazon Web Services, "Amazon DynamoDB Developer Guide." [Online]. Available: <https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/Introduction.html>
- [6] Amazon Web Services, "Amazon S3 User Guide." [Online]. Available: <https://docs.aws.amazon.com/AmazonS3/latest/userguide/Welcome.html>

- [7] Amazon Web Services, “Amazon Athena User Guide.” [Online]. Available: <https://docs.aws.amazon.com/athena/latest/ug/what-is.html>
- [8] Amazon Web Services, “Amazon EventBridge User Guide.” [Online]. Available: <https://docs.aws.amazon.com/eventbridge/latest/userguide/eb-what-is.html>
- [9] Amazon Web Services, “AWS Lambda Developer Guide.” [Online]. Available: <https://docs.aws.amazon.com/lambda/latest/dg/welcome.html>
- [10] T. Vu, “vnstock.” [Online]. Available: <https://github.com/thinh-vu/vnstock>
- [11] ExchangeRate-API, “API documentation.” [Online]. Available: <https://www.exchangerate-api.com/docs/overview>
- [12] Vercel, “Next.js documentation.” [Online]. Available: <https://nextjs.org/docs>
- [13] S. Ramírez, “FastAPI documentation.” [Online]. Available: <https://fastapi.tiangolo.com/>
- [14] OpenRouter, “API reference.” [Online]. Available: <https://openrouter.ai/docs>